

Task Management System.

Factory Method + Strategy, in pure Java.

A task management system for software teams, designed around two classic design patterns.

Abdul Rahman Malak
Mouhammad Houjeirat

What this presentation covers.

01	Introduction & problem definition	Abdul Rahman ~3 min
02	The design patterns: Factory Method + Strategy	Abdul Rahman + Mouhammad ~5 min
03	Architecture & implementation	Mouhammad ~3 min
04	Live demonstration	Both ~3 min
05	Results, evaluation & conclusion	Abdul Rahman ~2 min

Roughly 16 minutes of talk and demo, then questions. Both presenters take questions on the parts they led.

Introduction & problem definition.

The domain we chose, the objectives we set, and the two problems that shaped the whole design.

Why a task management system?

Software teams track many kinds of work. How that work is created and ordered is exactly where a design tends to decay over time.

The domain

A tracker for a software team: bugs, features, and documentation tasks, each moving through a shared lifecycle from open to done.

Our objectives

Support several task types and several orderings, and let new ones be added without editing existing, tested code.

The motivation

Two recurring causes of code decay in such a system map cleanly onto two well known design patterns from the course.

The solution in one line

One engine, three ways to run it, sixteen Java files, and zero external dependencies.

What the system must do.

FUNCTIONAL REQUIREMENTS

- Create bug, feature, and documentation tasks.
- Order the list by priority, deadline, or severity.
- Move tasks through a validated lifecycle.
- Filter the list by status and summarise it.

NON-FUNCTIONAL REQUIREMENTS

- Extensible: new types or orderings, no edits to existing code.
- Pure Java 8+, zero external dependencies.
- Runnable from the terminal: tests, console, and a GUI.
- Explainable: the design must hold up in a viva.

Two requirements drive the whole design: tasks are created in different ways, and the same list is ordered in different ways.

Tasks have different creation logic.

Each task type carries different fields and different defaults. Building them by hand couples every caller to the concrete classes.

```
new BugTask(title, priority, "MEDIUM", "");           // severity + steps
new FeatureTask(title, priority, 8, 5);                // effort + value
new DocumentationTask(title, priority, "API", "Developers");
```

The smell

Every entry point imports and names BugTask, FeatureTask, and DocumentationTask directly.

The consequence

Adding a fourth type means finding and editing every place that creates a task.

What we need

Create a task from a type identifier, without the caller naming a concrete class.

Ordering depends on the context.

The same task list is sorted differently at different moments. The list never changes; only the ordering rule does.

```
switch (sortMode) {  
    case DEADLINE: /* sort by due date    */ break;  
    case SEVERITY: /* bugs first, by rank */ break;  
    case URGENT:   /* sort by priority    */ break;  
}
```

The smell

One growing switch statement inside the manager carries every ordering rule.

The consequence

Each new context edits tested code, which breaks the Open/Closed principle.

What we need

Swap the ordering at runtime, and add new orderings without touching the manager.

The design patterns.

Two patterns chosen to match the two problems, plus a third that the task lifecycle gave us almost for free.

Matching patterns to problems.

We did not start from the patterns. We started from the two problems, and each one points to a single, well-fitted pattern.

Problem 1 · different creation logic	→	Factory Method Creational pattern	Decouples the caller from the concrete task classes.
Problem 2 · context-dependent ordering	→	Strategy Behavioral pattern	Makes the ordering a swappable object, chosen at runtime.
Bonus · the task lifecycle	→	State (in an enum) Behavioral pattern	Each status declares its own legal transitions.

Factory Method, and why it fits.

DEFINITION

Define a method for creating an object, and let each subclass decide which concrete class it produces.

WHY HERE

Every task type has its own construction rules and defaults. The caller should not have to know them.

ADVANTAGE

A new task type is a new class. Code that creates tasks never changes when a type is added.

REAL-WORLD

The same idea behind a document app that opens .pdf, .docx, or .txt through one Open action.

How we implemented it.

The manager keeps a registry of factories keyed by a string. The caller asks for a type by name and never sees a concrete class.

```
manager.createTask("BUG", "Login broken", "OAuth callback fails", 8);  
manager.createTask("FEATURE", "Dark mode", "Top user request", 5);
```

Abstract class, not an interface

Each subclass owns its defaults: a bug is MEDIUM severity, a feature is 8h effort and value 5, a doc targets developers.

createTaskWithDeadline() on the base

The base wraps createTask() and attaches a deadline once. That is Template Method, reused by all three factories.

Strategy, and why it fits.

DEFINITION

Define a family of interchangeable algorithms, encapsulate each one, and select between them at runtime.

WHY HERE

The same task list must be ordered differently in different contexts, while the app is running.

ADVANTAGE

Add or switch an ordering without touching the code that holds and manages the list.

REAL-WORLD

Like choosing a route in a maps app: fastest, shortest, or no tolls, computed over the same map.

How we implemented it.

Each ordering is a swappable object behind one interface. Changing the sort is a single setter call; the manager stays unaware of which is active.

```
manager.setPriorityStrategy(new SeverityFirstStrategy());  
// the next call to getPrioritizedTasks() returns the new order
```

An interface, not a raw Comparator

PriorityStrategy names the role. SeverityFirst also partitions bugs to the top, which a pairwise Comparator handles awkwardly.

A setter, not a constructor argument

Users change the sort mid-session. A setter avoids rebuilding the manager and copying every task into a new instance.

The same list, reordered live.

SEN3006 Task Manager

Demo Help

Create Task

Type: BUG Title: Priority: 3

Description: + Add Task

Sort by: Urgent First Filter: All Statuses

ID	Type	Title	Priority	Status	Deadline
1	BUG	Fix payment bug	5	OPEN	2026-05-01
5	FEATURE	Export CSV	4	OPEN	2026-05-10
4	DOCUMENTATION	Setup guide	3	OPEN	2026-04-20
2	FEATURE	Add search	2	OPEN	2026-06-15
3	BUG	UI glitch	1	OPEN	—

Selected Task Actions

→ IN_PROGRESS → REVIEW → DONE Block Unblock (→ OPEN) Delete

5 tasks | 5 OPEN | Strategy: UrgentFirstStrategy

Urgent First sort: priority 5 down to 1.

SEN3006 Task Manager

Demo Help

Create Task

Type: BUG Title: Priority: 3

Description: + Add Task

Sort by: Severity First Filter: All Statuses

ID	Type	Title	Priority	Status	Deadline
1	BUG	Fix payment bug	5	OPEN	2026-05-01
3	BUG	UI glitch	1	OPEN	—
5	FEATURE	Export CSV	4	OPEN	2026-05-10
4	DOCUMENTATION	Setup guide	3	OPEN	2026-04-20
2	FEATURE	Add search	2	OPEN	2026-06-15

Selected Task Actions

→ IN_PROGRESS → REVIEW → DONE Block Unblock (→ OPEN) Delete

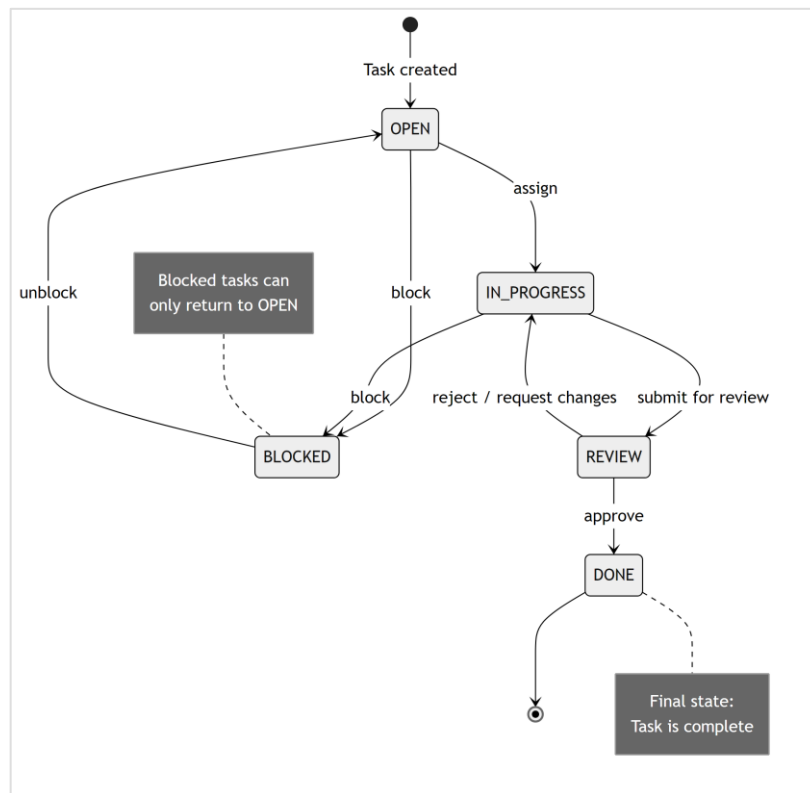
5 tasks | 5 OPEN | Strategy: SeverityFirstStrategy

Severity First sort: both bugs rise to the top.

Task Management System · SEN3006

14 / 28

The lifecycle is a state machine.



TaskStatus encodes its own transitions.

Each enum constant declares its legal next states through `canTransitionTo()`.

`AbstractTask.setStatus()` checks that table and throws on an illegal move.

The engine refuses to skip **OPEN** straight to **DONE**: the cycle must be walked.

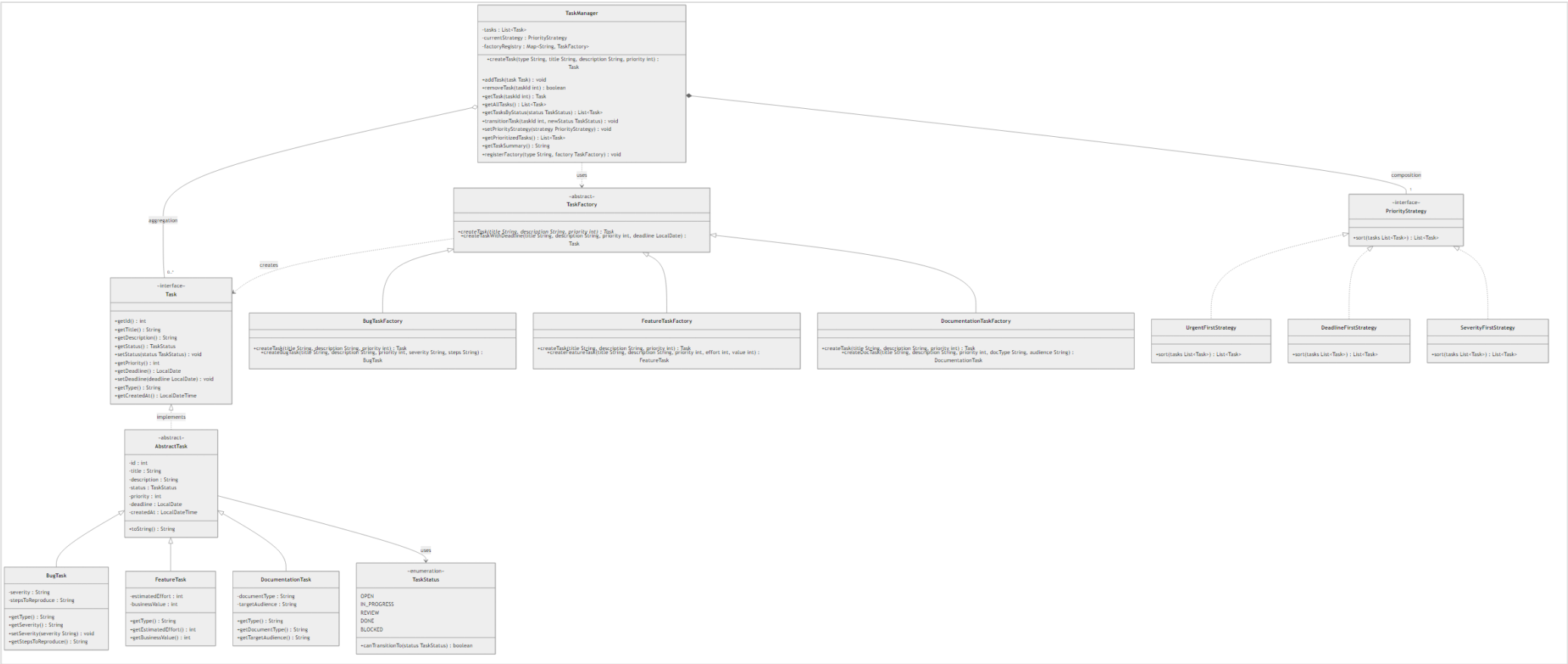
The State pattern, in a single enum file, with no extra classes.

Architecture & implementation.

How the design looks as classes, how a single request flows through it, and how it maps to Java and to SOLID.

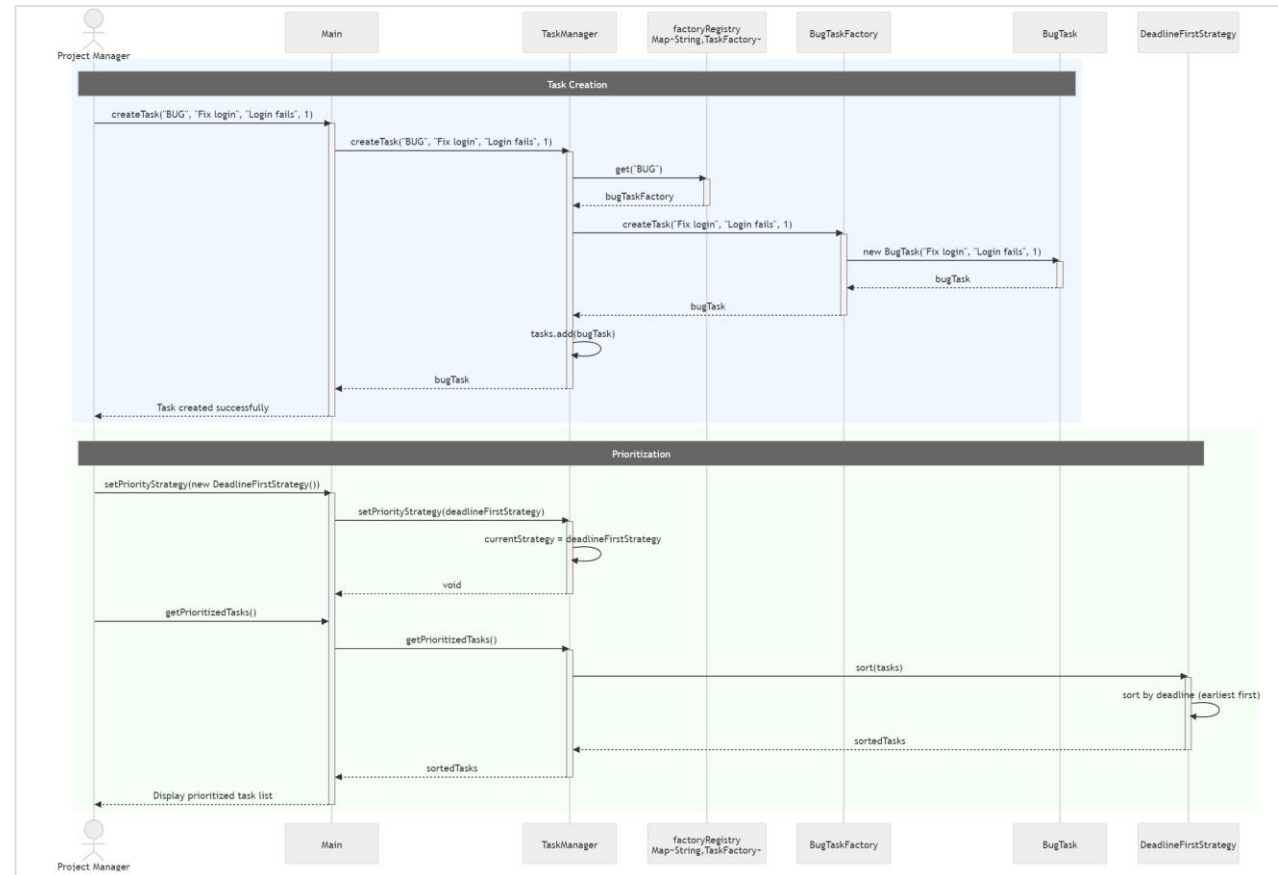
The class diagram.

Three layers: product (tasks) · pattern (factories + strategies) · coordination (the manager and entry points). Every arrow points toward an abstraction.



Creating a task, step by step.

The caller passes a type string. The manager finds the registered factory and returns a Task. No concrete class is ever named by the caller.



Sixteen files, three layers.

No package declarations, only java.util and java.time imported. The folders below mirror the three layers of the class diagram.

PRODUCT LAYER

- Task (interface)
- AbstractTask (base)
- BugTask · FeatureTask
- DocumentationTask
- TaskStatus (enum)

PATTERN LAYER

- TaskFactory (abstract)
- Bug / Feature / Doc factories
- PriorityStrategy (interface)
- UrgentFirst · DeadlineFirst
- SeverityFirst

COORDINATION & ENTRY

- TaskManager (coordinator)
- Main (automated tests)
- TaskManagementApp (console)
- gui.TaskManagerGUI (Swing)

All three entry points drive the same engine through its public API. 2 interfaces · 1 enum · 1 abstract · 12 concrete classes.

How the design meets SOLID.

- **S · Single responsibility**
Factories build, strategies sort, the manager coordinates. One reason to change each.
- **L · Liskov substitution**
Every factory works through the abstract reference; every strategy through the interface.
- **D · Dependency inversion**
The manager's fields are all interfaces and abstract types, never concretes.
- **O · Open / Closed**
A new task type or sort is a new file plus one line. Zero edits to existing classes.
- **I · Interface segregation**
PriorityStrategy declares one method; the Task interface stays minimal.
- **The throughline**
Abstractions sit in the middle of the system; concrete classes live at the edges.

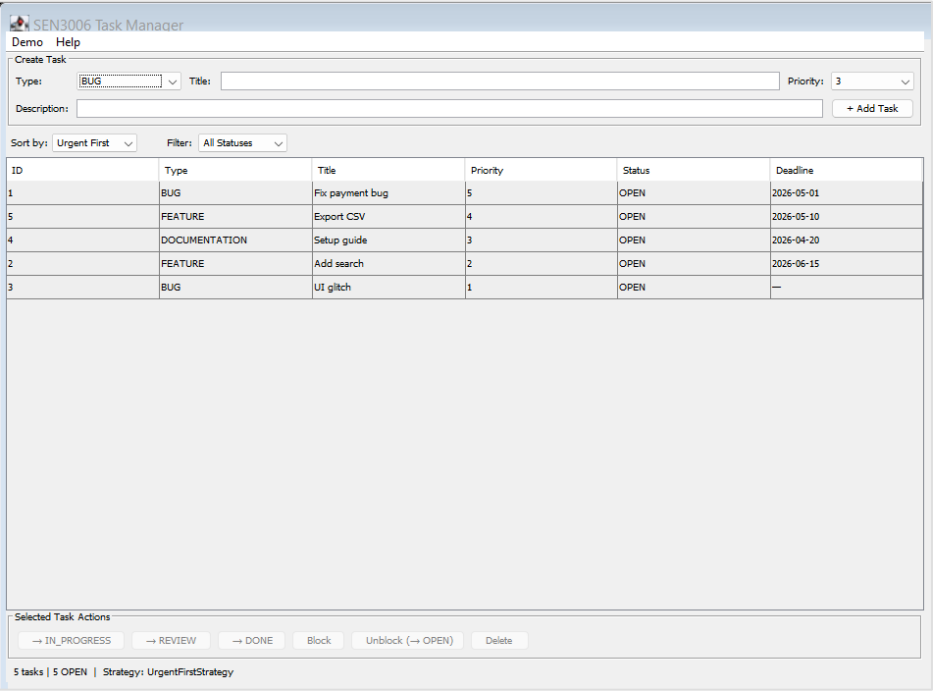
Live demonstration.

The compiled system, run three ways: the automated test suite, the interactive console, and the Swing GUI.

Running the system.

BUILD ONCE

```
javac -d bin src/main/java/*.java src/main/java/gui/*.java
```



1 · Automated tests

java -cp bin Main six sections, ends ALL TESTS PASSED.

2 · Open the GUI

java -jar TaskManagerGUI.jar then Demo, Load Strategy Demo.

3 · Strategy, live

switch Sort by: Urgent, Deadline, Severity. The table reorders, no recompile.

4 · Factory, live

add a task from the form. Type is BUG, FEATURE, or DOCUMENTATION.

5 · State machine

click a row: OPEN to DONE is refused, OPEN to IN_PROGRESS is allowed.

Validation lives in the engine, not the GUI. Every entry point shares the same Factory Method + Strategy core.

Results, evaluation & conclusion.

What the patterns delivered in practice, the honest limits of this implementation, and what we take away.

New features, without editing old code.

EXTENSION, IN NUMBERS

- New task type: 2 new files, 1 line of registration, 0 edits.
- New ordering: 1 new file, 1 setter call, 0 edits.
- Test 5 in Main asserts this Open/Closed property at runtime.

WHAT WORKED IN PRACTICE

- Documentation type was added late in under 30 minutes.
- The GUI dropdown proves the Strategy swap, live.
- The lifecycle enum caught two real bugs in development.
- Main doubles as a test harness, with no JUnit needed.

An honest look at the design.

Limitations

All data is in memory; nothing persists. Single user, no concurrency. Validation throws rather than collecting all errors.

Alternatives we weighed

Abstract Factory: rejected, there is only one product family. A raw Comparator: loses the named role and the stateful sort.

Future work

Persistence via Repository, tags via Specification, async dispatch, and a small REST layer over the same engine.

Being explicit about what is not built, and why, is part of the design. The engine is decoupled enough that each item above is an addition, not a rewrite.

What we set out to do, and what we learned.

What we achieved

A working task system where new types and new orderings cost no edits to existing, tested code.

The lesson

Naming a role, a factory or a strategy, makes the codebase explain itself to the next reader.

The patterns earned their place

Each was chosen to solve a concrete problem from section one, not to satisfy a checklist.

The bigger point

Design patterns are a shared vocabulary for keeping software open to change without fear.

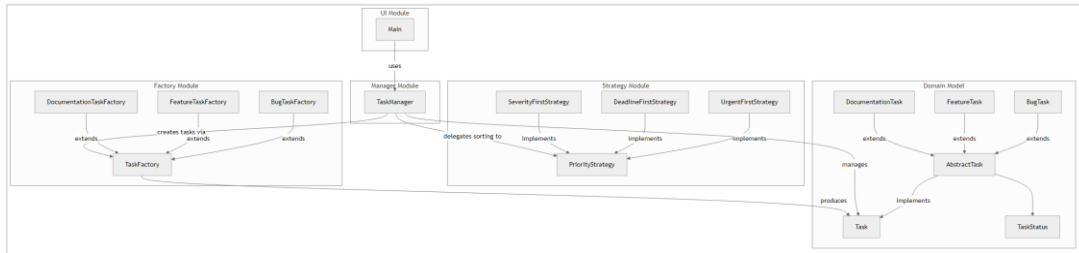
Thank you.

Questions welcome.

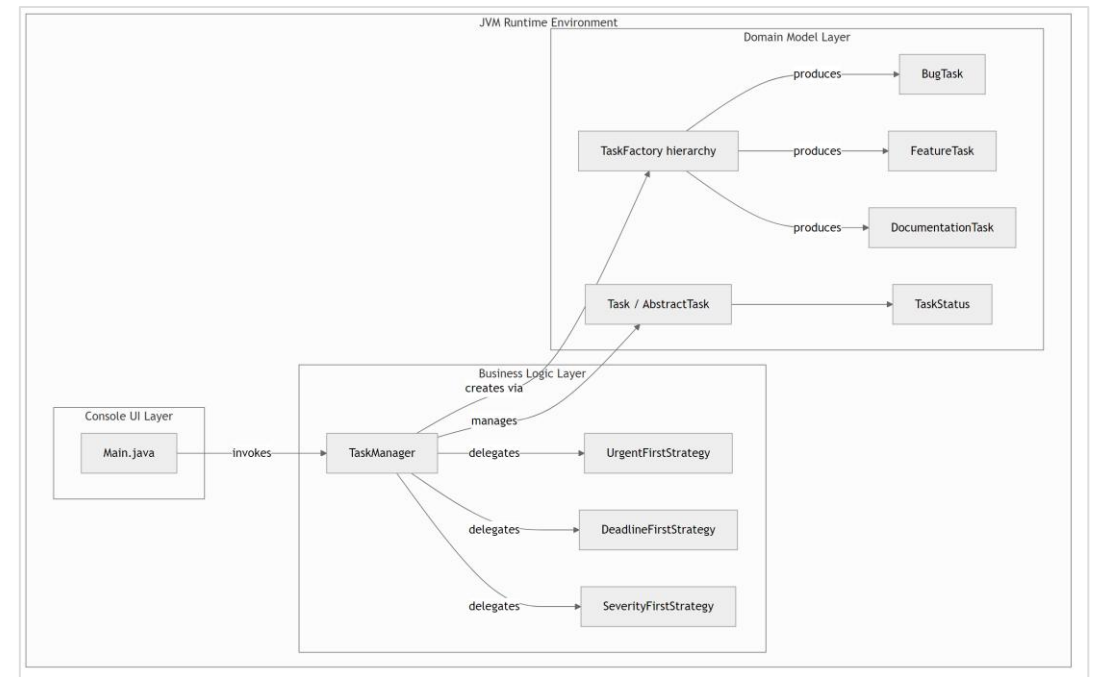
We can speak to any part: the problem, the patterns, the architecture, the code, or the demo.

SEN3006 Software Architecture · Abdul Rahman Malak · Mouhammad Houjeirat

Component & deployment views.



Component view: the three layers as modules.



Deployment view: one JVM, three entry points.